

SESSION 9 — FUNCTIONS

Before learning syntax, first understand WHY functions exist.

Imagine you write this:

```
print("Welcome") print("Welcome") print("Welcome")
```

Now imagine you need to print this 50 times in different places.

You would copy and paste the same code again and again.

This creates problems:

- Repetition
- Hard to maintain
- Hard to debug
- Messy code

A function solves this problem.

A function is: A reusable block of code that performs a specific task.

Instead of writing code again and again, you write it once and reuse it.

Real-life example: Think of a function like a machine. You give input. It processes. It gives output.

Basic Syntax of a Function

```
def function_name(): code
```

Important parts:

`def` → keyword that tells Python we are defining a function

`function_name` → name you give

`()` → parentheses (used to pass data later)

`:` → required colon

Indented block → code that belongs to function

✓ FIRST BASIC FUNCTION

```
def greet():  
    print("Welcome to Python")  
  
# Calling the function  
greet()
```

Welcome to Python

Start coding or [generate](#) with AI.

```
def Add(a = 3,b = 4):  
# def Add(a,b):  
    A = a + b  
    print(A)  
  
Add(3,5)  
Add(1,-5)  
Add(6.3,7.5)  
# Add(a,b)  
Add()
```

8
-4
13.8
7

```
A = 5  
# print(A)
```

Defining vs Calling

Defining a function: You are creating it.

Calling a function: You are using it.

If you define but never call, it will never execute.

✓ CALLING MULTIPLE TIMES

```
def greet():  
    print("Hello Student")  
  
greet()
```

```
greet()  
greet()
```

```
Hello Student  
Hello Student  
Hello Student
```

✓ Functions With Arguments

Sometimes functions need data.

Example: Greeting different people.

Instead of fixing name inside function, we pass name as input.

The input passed into a function is called: Argument

```
def greet(name):  
    print("Hello", name)  
  
greet("Rahul")  
greet("Anita")
```

```
Hello Rahul  
Hello Anita
```

Explanation:

- name is a parameter.
- "Rahul" is an argument.
- Parameter = variable inside function definition
- Argument = value passed while calling

✓ Return Statement

So far, functions only print.

But printing is not always enough.

Sometimes we want the function to: Calculate something And give back the result.

That is done using return.

```
def add(a, b):  
    result = a + b  
    print(result)  
  
value = add(5, 3)  
print("Sum:", value)
```

```
8  
Sum: None
```

```
def add(a, b):  
    result = a + b  
    print(result)  
    return result  
  
value = add(5, 3)  
print("Sum:", value)
```

```
8  
Sum: 8
```

```
# name1 = input("Enter name :")  
  
def name():  
    print(name1)  
name1 = input("Enter name :")  
name()
```

```
Enter name :rrr  
rrr
```

✓ Explanation:

- return sends value back.
- Without return, function gives None.

Important rule:

- After return, function stops executing.

```
def add(a, b):  
    return a + b  
add(5,3)
```

```
8
```

Printing inside function is limited.

Return allows:

- Store result in variable
- Use result later
- Combine functions
- Build large programs

Return makes functions powerful.

✓ Default Arguments

You can give default value to parameters.

If user does not pass value, default is used.

```
def greet(name="Student"):  
    print("Hello", name)
```

```
greet()  
greet("Rahul")
```

```
Hello Student  
Hello Rahul
```

Explanation:

- If no argument → uses "Student"
- If argument provided → overrides default

✓ Demo Function: calculate_discount(price, rate)

Goal:

- Take original price.
- Apply discount rate.
- Return final price.

Formula:

- `discount = price * rate`
- `final_price = price - discount`

```
def calculate_discount(price, rate):  
    discount = price * rate  
    final_price = price - discount  
    return final_price  
  
result = calculate_discount(1000, 0.10)  
print("Final Price:", result)
```

Final Price: 900.0

```
def calculate_discount(price, rate):  
    discount = price * rate  
    final_price = price - discount  
    return final_price  
  
a = int(input("Enter Price :"))  
f = int(input("Enter Discount :"))  
r = f/100  
result = calculate_discount(a, r)  
print("Final Price:", result)
```

Enter Price :100
Enter Discount :10
Final Price: 90.0

✓ Step-by-step explanation:

- `price = 1000`
- `rate = 0.10`
- `discount = 1000 * 0.10 = 100`
- `final_price = 1000 - 100 = 900`

Function returns 900.

Common Mistakes:

1. Forgetting parentheses when calling
2. Confusing parameter and argument
3. Forgetting return
4. Printing instead of returning

5. Indentation errors

Golden Rule:

Function definition does nothing until called.

✓ Function Practice Questions

1. Write a function called `greet` that prints the message `Welcome to Python` when called.
2. Write a function called `show_number` that takes a number as argument and prints the number.
3. Write a function called `square` that takes a number as input and prints its square.
4. Write a function called `add_numbers` that takes two numbers as arguments and prints their sum.
5. Write a function called `display_name` that takes a name as input and prints `Hello` followed by the name.
6. Write a function called `multiply` that takes two numbers as arguments and returns their product.
7. Write a function called `find_max` that takes two numbers as arguments and returns the larger number.
8. Write a function called `calculate_area` that takes the radius of a circle as argument and returns the area of the circle.
9. Write a function called `calculate_discount` that takes price and rate as arguments and returns the final price after discount.
10. Write a function called `check_even` that takes a number as argument and prints whether the number is even or odd.
11. Write a function called `average_numbers` that takes three numbers as arguments and returns their average.
12. Write a function called `find_largest` that takes a list of numbers as argument and returns the largest value in the list.
13. Write a function called `count_digits` that takes a number as argument and returns the number of digits in that number.

14. Write a function called `total_sales` that takes a list of sales values as argument and returns the total sales.
15. Write a function called `clean_product_name` that takes a string as argument and performs the following operations
 - convert the string to lowercase
 - remove extra spaces from beginning and end
 - replace any dash with a space
 - return the cleaned product name

```
a =5
def area(r):
    a = 3.14*r*r
    return a
b = int(input("Enter Radius :"))
print("Area of circle :",area(b))
```

```
Enter Radius : 1
Area of circle : 3.14
```

```
def area():
    r = int(input(" enter radiur : "))
    a = 3.14*r*r
    # return a
    print(a)
area()
# b = int(input("Enter Radius : "))
# print("Area of circle :",area(b))
```

```
enter radiur : 1
3.14
```

```
def area():
    r = int(input(" enter radiur : "))
    a = 3.14*r*r
    # return a
    print(a)
# r = int(input(" enter radiur : "))
print(r+2)
area()
# b = int(input("Enter Radius : "))
# print("Area of circle :",area(b))
```

```
3
```

```
def area():
    r = int(input(" enter radiur : "))
```

```
a = 3.14*r*r
# return a
print(a)
# r = int(input(" enter radiur : "))
print(r+2)
area()
# b = int(input("Enter Radius : "))
```

```
def ravi()

a = input()
prin
```

Start coding or [generate](#) with AI.

```
Welcome to Python
The number is: 10
Square: 25
Sum: 10
Hello Rahul
Product: 12
Larger number: 20
Area of circle: 78.5
Final price after discount: 900.0
Number is Odd
Average: 20.0
Largest value: 20
Number of digits: 5
Total sales: 7500
Cleaned product: samsung mobile
```

Start coding or [generate](#) with AI.